Take-Home Assignment

Reflection AI

Context

You are a Forward Deployed Engineer at Reflection AI, engaged with Apex Manufacturing, a mid-size industrial equipment manufacturer. The client held a discovery session identifying their primary pain point: procurement planning for production orders is slow, error-prone, and doesn't scale as their product portfolio grows.

You will meet with Apex Manufacturing's Head of Operations in a few days to present a working interim prototype. This is a milestone check-in within an ongoing project: focus on demonstrating initial capabilities, surfacing open questions, and confirming direction.

Problem

Apex Manufacturing's procurement planners currently work through a manual process for every production order. They look up the bill of materials to identify required components, check current inventory levels, find suitable suppliers while complying with internal policies, and place purchase orders — all while accounting for lead times, inspection requirements, and management memos that update policy provisions over time.

This process is a major bottleneck as order volume grows and policies become more complex.

Solution

Apex wants an **autonomous AI procurement agent** that examines the current production schedule, identifies what materials are needed, and places purchase orders — all while respecting procurement policies and management directives. The system operates without human prompting: it reads the state of the world and acts.

Your Task

Build a working prototype of this procurement agent. The result should be a Python application that can run against any scenario database and produce defensible procurement actions.

How It Works

Each scenario is a self-contained SQLite database representing a snapshot of Apex Manufacturing's state. The policies and memos are shared across all scenarios (they're company-wide documents).

Your agent receives the path to the given scenario sqlite file as its only argument:

```
None
python3 agent.py --scenario <scenario.sqlite>
```

It should examine the production schedule, determine what procurement actions are needed, and write its decisions to two tables in the database:

purchase_orders — each row is an order the agent places: component_id, supplier_id, quantity, unit_price, order_date, expected_delivery_date, and a rationale field explaining the decision.

alerts — each row is a natural-language description of a problem or recommendation (e.g., "Cannot meet May 20 start date for MFG-5030: neodymium magnet lead time exceeds available window.").

> *We will also run your agent against held-out scenarios not included here. Design to generalize.*

Technical Requirements

Structure your tech stack so that Reflection's open model can be slotted in once available. Assume Reflection models will be compatible with any industry-standard framework supporting leading open models today. You may not use proprietary orchestration frameworks locked to a single model provider.

Submission and Presentation

Submit a zip file with your Python project and a README. You will present in a 30-minute session role-playing as an FDE presenting to the customer's Head of Operations. Plan ~15 minutes of presentation/demo and ~15 minutes of Q&A.

Provided Data

Apex Manufacturing has provided data for their US industrial equipment division. In practice, the customer has dozens of product lines.

Database Schema

Six scenario databases (`data/scenarios/*.sqlite`) share this schema:

Table	Description
`scenario_config`	Current date and scenario description
`products`	Finished goods catalog (this is what Apex produces)
`components`	Raw materials and parts, with hazmat flags and certification requirements
`bom`	Product-component mapping, with quantity per unit (flat, single level)
`suppliers`	Supplier directory: certifications, sustainability ratings, relationship tiers
`supplier_catalog`	Supplier-component pricing, lead times, minimum order quantity
`inventory`	Current stock levels by component
`production_schedule`	Upcoming production orders with material deadlines and customers
`purchase_orders`	Pre-existing orders (if any) — **your agent adds new rows here**
`alerts`	Empty — **your agent writes problems and recommendations here**

> *The BOM is flat (single level): each finished good lists its components directly. Some components are shared across multiple products.*

Policy and Memos

One procurement policy document (`data/policies/`) and three management memos (`data/memos/`) that update specific policy provisions. Each memo is dated — when a memo conflicts with a base policy, the memo takes precedence.

Scenarios

Six scenarios ranging from straightforward single-product procurement to complex situations with shared component contention, pre-existing purchase orders, and tight timelines. A manifest (`data/scenarios/manifest.json`) describes each one.

What We're Looking For

We value thoughtful design and honest assessment of limitations over a polished but shallow demo.

This exercise does not perfectly specify all behavior that a procurement agent should implement (far from it!). Your job is to make decisions about system design based on what you know about the problem, while communicating questions and tradeoffs to the hypothetical customer.

Good luck!